Formalizing Bounded Context Selection: Runtime Signals and Optimizer Extensions for LLM Coding Agents

Bounded Reasoning Environment Core Research Group

May 23, 2026

Abstract

Large language model agents increasingly operate over software repositories whose full state exceeds practical context limits. Naive long-context loading increases latency and cost while exposing models to attention degradation and irrelevant middle-context material. This paper formalizes the Bounded Reasoning Environment (BRE), a task-first context runtime that partitions a repository into Active, Watching, and Background trays, then selects concrete file representations under token and loaded-file limits. We separate the implemented BRE runtime from proposed optimizer extensions. The current implementation uses deterministic task contracts, language-aware dependency discovery, a first-class signal vector, configurable priority aggregation with max-norm as the default, greedy bounded selection, project-aware diagnostics, and adaptive weight boost/decay. We then formulate a future optimizer as a multidimensional multi-choice knapsack problem over representation states and outline a dual-relaxed greedy approximation and momentum-backed projected feedback update. Evaluation evidence from the current implementation includes an 8/8 synthetic suite pass, an 8/8 real-repository suite pass, 0.90 recall on a 36-file Visual Studio Code stress task, and 1.00 recall on a 26-file OpenNext Cloudflare stress task.

1 Introduction

Modern software agents often fail before code generation: they reason over the wrong repository boundary. A feature request may implicate source files, manifests, framework configuration, generated types, deployment descriptors, and tests. Treating the repository as unstructured text and retrieving by semantic proximity alone misses compiler-level structure. Conversely, placing all available files into the prompt creates cost, latency, and attention problems, including the long-context degradation described by Liu et al. [1].

BRE treats context as a bounded engineering resource. The core thesis is not that every file should be available to the model, but that every inclusion, compression, and omission should be explicit, inspectable, and connected to task or compiler signals. This framing differs from conventional retrieval-augmented generation [2]: code context selection is constrained not only by topical relevance, but also by build graphs, imports, manifests, type contracts, and diagnostic feedback.

Contribution. This paper makes two claims at different maturity levels:

- **Implemented runtime.** BRE currently implements deterministic trays, signal scoring with selectable aggregation policies, greedy token bounding, project-aware diagnostics, structured run reports, file decision traces, JavaScript alias handoff, and adaptive weight boost/decay.

- **Proposed optimizer.** BRE can be extended into a formal multidimensional multi-choice knapsack optimizer with dual relaxation and momentum-backed projected feedback. These pieces are formulated here but not claimed as implemented in the current engine.

2 Current BRE Runtime

Let a repository be a finite file set

$$\mathcal{C} = \{f_1, f_2, \ldots, f_N\}. \tag{1}$$

The current runtime receives a task contract consisting of a goal, acceptance criteria, assumptions, non-goals, and explicitly active files. It emits an agent payload:

$$\mathrm{BRE}(R, C, O) \to P, \tag{2}$$

where R is the repository root, C is the contract, O is runtime options, and P contains context trays, diagnostics, and an optional structured report. The current implementation exposes this boundary as a reusable runtime rather than only as a command-line interface.

Definition 1 (Task Contract)**.** *The task contract C is a tuple*

$$C = (g, A_C, Q, S, N_C) \tag{3}$$

where g is the natural-language goal, A_C is the explicit Active file set, Q is acceptance criteria, S is assumptions, and N_C is non-goals. The contract is not treated as model conversation history; it is an operating packet that constrains context selection.

Definition 2 (Run Report)**.** *The optional run report is a nested tuple*

$$\mathcal{R} = (R, D, K, W, \mathcal{T}) \tag{4}$$

where R is the repository root, D is diagnostic telemetry, K is context-build telemetry, W is weight-feedback telemetry, and $\mathcal{T}$ is the set of file decision traces. This separation keeps prompt payload content distinct from explainability metadata.

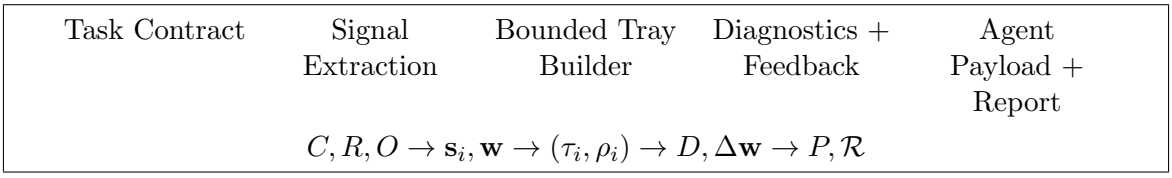


Figure 1: Implemented BRE runtime architecture. The current system is deterministic through signal extraction and tray assembly; diagnostics optionally update weights after reportable verification signals.

2.1 Tray Role and Representation State

BRE separates a file's *tray role* from its *representation state*. This distinction is central because a Watching file is not necessarily skeletonized.

Definition 3 (Tray Role)**.** *Each file is assigned one role:*

$$\tau_i \in \{\mathit{Active}, \mathit{Watching}, \mathit{Background}\}. \tag{5}$$

Active files are explicitly named by the task and loaded exactly. Watching files are loaded into the prompt as supporting context. Background files are omitted from the prompt body but retained as inventory and possible evidence of ejection.

Definition 4 (Representation State)**.** *Each file also has a representation:*

$$\rho_i \in \{\mathit{Full}, \mathit{Skeleton}, \mathit{Omitted}\}. \tag{6}$$

Full means exact file content is included. Skeleton means a lossy structural representation is included. Omitted means the body is excluded. In the current runtime, Active files use Full, Watching files may use Full or Skeleton, and Background files use Omitted.

Definition 5 (Context Configuration)**.** *For each file f_i, BRE emits a pair (τ_i, ρ_i). The tray role controls whether and where the file appears in the agent context. The representation state controls the amount of file content included. These variables are related but not identical: $\tau_i =$ Watching permits either $\rho_i =$ Full or $\rho_i =$ Skeleton.*

The runtime report records this distinction through file decision traces:

$$(\mathit{path}_i, \tau_i, \rho_i, \mathit{score}_i, \mathit{signals}_i). \tag{7}$$

The prompt context file itself remains content-focused; explainability belongs to the report.

Lemma 1 (Lean Payload Property)**.** *When reports are disabled, prompt context files contain only path, optional content, and optional skeleton markers. Signal labels, scores, and tray explanations are omitted from the prompt payload.*

Proof. In the implemented runtime, file decision traces are emitted only through the optional report field $\mathcal{T}$. The context file structure does not contain a signal field. Therefore, disabling $\mathcal{R}$ removes explainability metadata while preserving the task contract, selected file contents, and diagnostics. □

3 Signal Vector

For each candidate file f_i, BRE computes a structured signal vector:

$$\mathbf{s}_i = \begin{bmatrix} s_{i,\text{ast}} \\ s_{i,\text{symbol}} \\ s_{i,\text{manifest}} \\ s_{i,\text{git}} \\ s_{i,\text{proximity}} \\ s_{i,\text{task}} \\ s_{i,\text{critical}} \end{bmatrix}. \tag{8}$$

The implemented labels are direct AST/import dependency, symbol match score, active manifest proximity, git dirty state, active parent proximity, task path term match, task-term plus AST match, and critical configuration proximity.

Let $\mathbf{w}$ be the current heuristic weight vector. The default implemented priority score uses a max-norm style aggregator:

$$R_\infty(f_i \mid \mathbf{w}) = \max_j s_{i,j} w_j. \tag{9}$$

This design intentionally lets one strong structural reason keep a file hot. It is conservative for compiler-sensitive work because a single import edge or manifest boundary can be more important than many weak semantic hints.

The engine also exposes additive and p-norm aggregators for controlled comparisons:

$$R_1(f_i \mid \mathbf{w}) = \sum_j s_{i,j} w_j \tag{10}$$

and

$$R_p(f_i \mid \mathbf{w}) = \left(\sum_j (s_{i,j} w_j)^p \right)^{1/p}. \tag{11}$$

These policies reuse the same signal vector and report traces, allowing benchmarks to test whether weak signals should compound or whether a single strong signal should dominate.

4 Implemented Greedy Bounded Selection

BRE currently builds the context tray in deterministic phases:

1. Lock Active files from the task contract.
2. Discover Watching candidates through imports, symbols, manifests, dirty files, active-parent proximity, task terms, and critical files.
3. Compute a score and signal trace for each candidate.
4. Estimate token cost for Active and Watching files.
5. Skeletonize large or cold candidates when budget pressure requires it.
6. Eject the coldest Watching candidates to Background until token and loaded-file constraints are satisfied.
7. Backfill high-value exact files when budget remains.

Let B be the token budget and M the maximum loaded-file count. The implemented runtime enforces:

$$\sum_{\tau_i \in \{\text{Active},\text{Watching}\}} T(f_i, \rho_i) \le B \tag{12}$$

and

$$|\{i : \tau_i \in \{\text{Active}, \text{Watching}\}\}| \le M. \tag{13}$$

The current algorithm is deterministic and stable under path-score ties. It is not yet a full integer-programming solver.

5 Proposed MMKP Optimizer

The natural optimization model is a multidimensional multi-choice knapsack problem [3]. For each file f_i, choose exactly one representation state $c \in \{0, 1, 2\}$ corresponding to Omitted, Skeleton, and Full:

$$x_{i,c} \in \{0, 1\}, \quad \sum_{c=0}^{2} x_{i,c} = 1. \tag{14}$$

Let $T(f_i, c)$ be token cost and $L(c)$ be loaded-file cost, where $L(0) = 0$ and $L(1) = L(2) = 1$. Let $V(f_i, c \mid \mathbf{w})$ be utility:

$$V(f_i, c \mid \mathbf{w}) = \begin{cases} 0 & c = 0, \\ \alpha_i R(f_i \mid \mathbf{w}) & c = 1, \\ R(f_i \mid \mathbf{w}) & c = 2. \end{cases} \tag{15}$$

Here $\alpha_i \in (0, 1)$ models information retained by skeletonization. The proposed exact problem is:

$$\max_{x} \sum_{i=1}^{N} \sum_{c=0}^{2} V(f_i, c \mid \mathbf{w}) x_{i,c} \tag{16}$$

subject to

$$\sum_{i=1}^{N} \sum_{c=0}^{2} T(f_i, c) x_{i,c} \leq B, \tag{17}$$

$$\sum_{i=1}^{N} \sum_{c=0}^{2} L(c) x_{i,c} \leq M, \tag{18}$$

and Active locks $x_{a,2} = 1$ for f_a in the declared Active set.

5.1 Aggregation Policies

The current runtime implements R_∞, R_1, and R_p as selectable scoring policies. In the proposed optimizer, these become interchangeable utility aggregators:

$$R_1(f_i \mid \mathbf{w}) = \sum_j s_{i,j} w_j \tag{19}$$

and

$$R_p(f_i \mid \mathbf{w}) = \left(\sum_j (s_{i,j} w_j)^p \right)^{1/p}. \tag{20}$$

The key future step is not exposing the formulas themselves, but using benchmark results to decide which aggregator should drive MMKP utility under token and loaded-file constraints.

5.2 Dual-Relaxed Greedy Approximation

Relaxing $x_{i,c}$ to $[0, 1]$, define positive surrogate multipliers λ_1, λ_2 for token and file budgets. The incremental upgrade ratio from state $c - 1$ to c is:

$$\Delta_{i,c}(\lambda) = \frac{V(f_i, c) - V(f_i, c-1)}{\lambda_1 [T(f_i, c) - T(f_i, c-1)] + \lambda_2 [L(c) - L(c-1)] + \delta}. \tag{21}$$

For fixed λ, sorting all non-dominated transitions by $\Delta_{i,c}$ costs $O(N \log N)$ because there are at most two upgrades per file. Greedy insertion then costs $O(N)$. Searching the two-dimensional multiplier space gives a practical approximation path, but this is future optimizer work rather than current runtime behavior.

6 Adaptive Feedback

BRE uses diagnostics as learning signals, but only after filtering ignored diagnostics. Let $\mathbf{w}^{(t)}$ be the heuristic weight vector at run t and $\mathbf{w}^{(0)}$ the default baseline.

6.1 Implemented Boost and Decay

When diagnostics are clean, implemented weights decay toward defaults:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \gamma(\mathbf{w}^{(t)} - \mathbf{w}^{(0)}), \tag{22}$$

implemented discretely with bounded integer steps. This prevents saturation.

When a diagnostic implicates a Background file that has a decision trace, BRE boosts only the weights for signals that actually fired for that file. If the file has no trace, the engine records a signal coverage gap instead of blindly increasing unrelated weights. Thus, a miss caused by an AST dependency does not inflate git-dirty or task-term weights.

6.2 Proposed Momentum Extension

A future optimizer can frame feedback as projected subgradient descent. Let E_t be omitted files attributed to diagnostics, and define a hinge loss:

$$L^{(t)}(\mathbf{w}) = \max\left(0, C^{(t)} - \frac{1}{|E_t|}\sum_{e \in E_t} R(f_e \mid \mathbf{w})\right). \tag{23}$$

With velocity $\mathbf{v}^{(t)}$ and momentum $\beta \in [0, 1)$:

$$\mathbf{v}^{(t+1)} = \beta\mathbf{v}^{(t)} + (1-\beta)\mathbf{g}^{(t)}, \tag{24}$$

$$\mathbf{w}^{(t+1)} = \Pi_{\mathcal{W}}\left[\mathbf{w}^{(t)} - \eta^{(t)}\mathbf{v}^{(t+1)}\right]. \tag{25}$$

This is a proposed stability improvement. It is not part of the implemented engine in this pass.

7 Evaluation Evidence

The current BRE implementation is evaluated through deterministic context-selection suites and real-repository stress tasks. The measurements below are evidence of current runtime behavior, not proof of optimality. All measurements in this section were produced by the repository-local benchmark harness after the signal-report refactor.

The benchmark commands were:

```
cargo fmt --all --check
cargo test --quiet
cargo clippy --quiet --all-targets --all-features -- -D warnings
```

Suite	Target	Result	Precision	Notes
Synthetic suite	Local generated tasks	8/8	N/A	Boundary cases
Real repo suite	Open-source tasks	8/8	N/A	Oracle files
VS Code stress task	Keybinding resolver	0.90 recall	0.25	36 selected files
OpenNext Cloudflare	Build pipeline	1.00 recall	0.54	26 selected files

Table 1: Implemented BRE context-selection evidence used by this paper.

```
python3 benchmarks/run.py
python3 benchmarks/run_real_repos.py --bre-timeout 300
python3 benchmarks/run_real_repos.py \
  --task vscode_keybinding_resolver --bre-timeout 300
python3 benchmarks/run_real_repos.py \
  --task opennext_cloudflare_build_pipeline --bre-timeout 300
python3 benchmarks/run_scoring_policies.py --bre-timeout 300
```

The full real-repository run passed 8/8 tasks. The VS Code case is the primary stress benchmark because it forces the engine through a large TypeScript monorepo with broad fixture directories, nested platform layers, high byte pressure, and file-count pressure. The OpenNext Cloudflare task tests framework and build-pipeline discovery in a modern JavaScript/TypeScript repository.

The scoring-policy matrix passed 24/24 real-repository policy runs across max-norm, additive, and p-norm aggregation. Additive and p-norm preserved recall on every task and reduced the VS Code stress tray from 36 to 34 loaded files while keeping recall at 0.90. The default runtime policy remains max-norm until agent-outcome trials show that a different aggregator improves patch correctness or verification pass rate.

8 Benchmark Appendix

Table 2 lists the full real-repository suite result used as the paper's current empirical evidence. The suite is intentionally context-selection focused: it scores whether BRE places oracle files into Active or Watching before any LLM patching begins.

Task	Repository	Recall	Precision	Files
ggez_input_keyboard	ggez/ggez	1.00	0.17	12
macroquad_prelude_input	not-fl3/macroquad	1.00	0.33	6
avian_collision_layers	avianphysics/avian	1.00	0.20	10
tetra_keyboard_input	17cupsofcoffee/tetra	1.00	0.18	11
boardgameio_core_reducer	boardgameio/boardgame.io	1.00	0.30	20
valence_command_graph	valence-rs/valence	1.00	0.35	20
vscode_keybinding_resolver	microsoft/vscode	0.90	0.25	36
opennext_cloudflare_build_pipeline	opennextjs/cloudflare adapter	1.00	0.54	26

Table 2: Full real-repository context-selection suite after the signal-report refactor.

To guard the report layer itself against drift, the repository includes stable report snapshots for the VS Code and OpenNext Cloudflare stress tasks. These snapshots normalize machine-specific paths while preserving commits, counts, budgets, selected paths, and file decision traces.

9 Related Work

RAG systems retrieve textual evidence for generation [2], but source code context requires compiler and repository structure that semantic similarity alone may miss. Prompt compression systems such as LLMLingua [4] reduce prompt cost, while BRE decides which physical repository artifacts should enter the prompt and in what representation.

Program synthesis work such as neuro-symbolic synthesis [5] targets code generation from examples or specifications. BRE does not synthesize programs directly; it prepares a bounded, inspectable repository state for an external coding agent. Reinforcement-learning-based agent research [6] optimizes multi-turn behavior; BRE instead emphasizes deterministic context assembly with optional future feedback optimization.

10 Limitations and Future Work

Current BRE scoring remains heuristic even though max-norm, additive, and p-norm aggregators are now selectable. The engine does not yet implement MMKP solving, dual multiplier search, or momentum-backed feedback. Skeletonization remains language-dependent and can be improved by using syntax-aware pruning consistently across supported languages. Evaluation evidence demonstrates useful behavior on current suites, but broader agent-outcome trials are needed: patch correctness, verification pass rate, changed-file precision, token cost, wall time, and false-success rate.

11 Conclusion

BRE reframes LLM coding context as a bounded optimization problem over files, signals, representations, diagnostics, and budgets. The current implementation already provides deterministic trays, signal traces, greedy budget enforcement, diagnostics, and adaptive boost/decay. The formal model presented here gives the next architecture target: a reusable optimizer that can compare scoring policies, solve a principled multi-choice budget problem, and learn from compiler feedback without saturating or hiding attribution gaps.

References

[1] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.

[2] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, 2020.

[3] Md Mostofa Akbar, Eric G. Manning, Gholamali C. Shoja, and Shahadat Khan. Heuristic solutions for the multiple-choice multi-dimension knapsack problem. In *International Conference on Computational Science*, pages 659–668, 2001.

[4] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LLMLingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*, pages 13358–13376, 2023.

[5] Emilio Parisotto, Abdel rahman Mohamed, Rishabh Singh, Lihong Li, Dengyong Zhou, and Pushmeet Kohli. Neuro-symbolic program synthesis. In *International Conference on Learning Representations*, 2017.

[6] Zhepei Wei, Wenlin Yao, Yao Liu, Weizhi Zhang, Qin Lu, Liang Qiu, Changlong Yu, Puyang Xu, Chao Zhang, Bing Yin, Hyokun Yun, and Lihong Li. WebAgent-R1: Training web agents via end-to-end multi-turn reinforcement learning. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*, 2025.